

Interpreter Pattern

- Role-Based Access Control Rule Interpreter
- CSE 3206 Software Engineering Sessional • Group 4

Category & Intent

- Behavioral Design Pattern
- Represents a small grammar using expression objects and interprets expressions according to that grammar.

Real-World Problem

- Flexible access rules are required.
- Examples: ADMIN AND EDITOR; EDITOR OR MODERATOR; ADMIN AND (EDITOR OR MODERATOR).
- Many hard-coded conditions become difficult to maintain as rules grow.

Our Solution

- Represent rules as expression objects.
- Build an expression tree.
- Evaluate recursively using a Context.

Participants

- Expression → abstract expression
- RoleExpression → terminal expression
- AndExpression / OrExpression → non-terminal expressions
- Context → user roles
- Parser / Client → creates the expression structure

UML / Structure

- Expression is the common abstraction.
- Terminal expressions represent roles.
- Non-terminal expressions combine other expressions.
- UML source and PNG are in docs/.

Expression Tree

- ADMIN AND (EDITOR OR MODERATOR)
- AND
- /\
- ADMIN OR
- /\
- EDITOR MODERATOR

Execution Flow

- User roles → Context
- Rule text → Parser
- Parser → Expression Tree
- interpret(Context) → recursive evaluation
- TRUE → Access Granted | FALSE → Access Denied

Live Demonstration

- Select ADMIN + EDITOR.
- Enter: ADMIN AND (EDITOR OR MODERATOR)
- Evaluate → ACCESS GRANTED
- Then select ADMIN only and test EDITOR OR MODERATOR → ACCESS DENIED.

Code Review

- Naming convention
- SOLID principles
- Readability and object interaction
- Pattern correctness and reusability
- Exception handling
- Documentation, maintainability, efficiency

Advantages & Limitations

- Advantages: composability, extensibility, separation of responsibilities.
- Limitations: more classes for larger grammars, extra complexity, recursive evaluation overhead.

Conclusion

- Interpreter is suitable for a small rule language.
- This project demonstrates terminal/non-terminal expressions, Context, expression trees, recursive interpretation, UML, testing, and a web demo.